

JAVA INHERITANCE PRACTICAL RECORD

Five Separate Inheritance Programs

EXPERIMENT 11 — SINGLE INHERITANCE

Experiment No	11
Date	17/09/2026
Title	Single Inheritance

■ Objective / Aim

To write a Java program to demonstrate single inheritance, where one child class inherits the properties and methods of a single parent class.

■ System Requirements

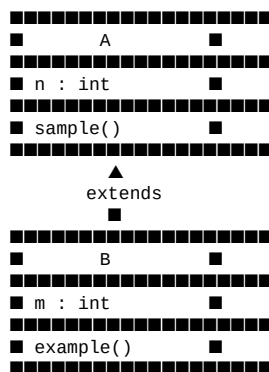
Hardware: Standard PC/Laptop, Core i3/i5 Processor, 4GB/8GB RAM.

Software: Windows/Linux/macOS, JDK 17/JDK 21, VS Code/Eclipse/IntelliJ IDEA/NetBeans.

■ Algorithm / Logic Description

1. Create a parent class A.
2. Declare the integer variable n.
3. Define sample() to assign n and calculate its square.
4. Create child class B extending A.
5. Declare m in class B and define example().
6. Calculate the sum of m and n.
7. Create an object of B.
8. Call sample() and example() and display the results.

■ UML Class Diagram



■ Source Code

```
class A{
    int n;
    void sample(int i)
    {
        System.out.println("This is from class A");
        n=i;
        System.out.println("The square of n is : "+(n*n));
    }
}
class B extends A{
    int m;
    void example(int j){
        System.out.println("This is from class b");
        m=j;
        System.out.println("The sum of m and n is : "+(m+n));
    }
}
public class SingleInheritance{
    public static void main(String[] args) {
        B obj=new B();
        obj.sample(4);
        obj.example(5);
    }
}
```

■ Compilation & Execution Commands

```
Save the file exactly as: SingleInheritance.java  
javac SingleInheritance.java  
java SingleInheritance.java
```

■■ Output Screen

```
This is from class A  
The square of n is : 16  
This is from class b  
The sum of m and n is : 9
```

■ Result

Thus, the Java program demonstrating single inheritance was successfully implemented and executed.

EXPERIMENT 12 — MULTILEVEL INHERITANCE

Experiment No	12
Date	17/09/2026
Title	Multilevel Inheritance

■ Objective / Aim

To write a Java program to demonstrate multilevel inheritance, where a class inherits from a derived class.

■ System Requirements

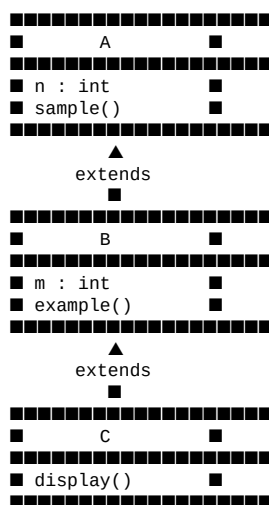
Hardware: Standard PC/Laptop, Core i3/i5 Processor, 4GB/8GB RAM.

Software: Windows/Linux/macOS, JDK 17/JDK 21, VS Code/Eclipse/IntelliJ IDEA/NetBeans.

■ Algorithm / Logic Description

1. Create parent class A and declare n.
2. Define sample() in A.
3. Create class B extending A and declare m.
4. Define example() in B.
5. Create class C extending B.
6. Define display() in C.
7. Create an object of C.
8. Call sample(), example(), and display().

■ UML Class Diagram



■ Source Code

```
class A{
    int n;
    void sample(int i)
    {
        System.out.println("This is from class A");
        n=i;
        System.out.println("The square of n is : "+(n*n));
    }
}
class B extends A{
    int m;
    void example(int j){
        System.out.println("This is from class b");
        m=j;
        System.out.println("The sum of m and n is : "+(m+n));
    }
}
class C extends B{
    void display(){
        System.out.println("This is from class c");
        System.out.println("The product of m and n is : "+(m*n));
    }
}
```

```
    }  
}  
public class Multilevel {  
    public static void main(String[] args) {  
        C obj=new C();  
        obj.sample(2);  
        obj.example(5);  
        obj.display();  
    }  
}
```

■ Compilation & Execution Commands

Save the file exactly as: Multilevel.java
javac Multilevel.java
java Multilevel.java

■■ Output Screen

This is from class A
The square of n is : 4
This is from class b
The sum of m and n is : 7
This is from class c
The product of m and n is : 10

■ Result

Thus, the Java program demonstrating multilevel inheritance was successfully implemented and executed.

EXPERIMENT 13 — HIERARCHICAL INHERITANCE

Experiment No	13
Date	17/09/2026
Title	Hierarchical Inheritance

■ Objective / Aim

To write a Java program to demonstrate hierarchical inheritance, where multiple child classes inherit from a single parent class.

■ System Requirements

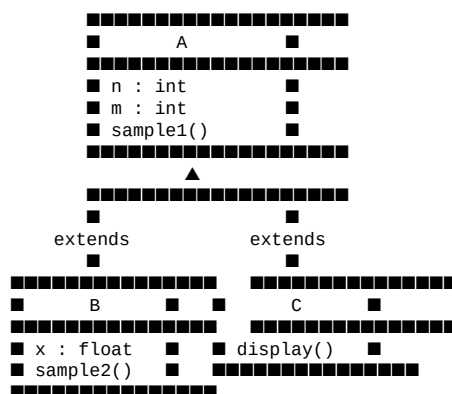
Hardware: Standard PC/Laptop, Core i3/i5 Processor, 4GB/8GB RAM.

Software: Windows/Linux/macOS, JDK 17/JDK 21, VS Code/Eclipse/IntelliJ IDEA/NetBeans.

■ Algorithm / Logic Description

1. Create parent class A with n and m.
2. Define sample1() to initialize n and m and display their sum.
3. Create child class B extending A and define sample2().
4. Create child class C extending A and define display().
5. Create objects of B and C.
6. Call the methods and display the results.

■ UML Class Diagram



■ Source Code

```
class A{
    int n;
    int m;
    void sample1(int i,int j)
    {
        n=i;
        m=j;
        System.out.println("This is from Super class ");
        System.out.println("The sum of m and n is : "+(m+n));
    }
}
class B extends A{
    float x;
    void sample2(float l)
    {
        x=l;
        System.out.println("This is from sub class 1");
        System.out.println("The sum is : "+(x+m+n));
    }
}
class C extends A{
    void display()
    {
        System.out.println("This is from sub class 2");
        System.out.println("The product of m and n is : "+(m*n));
    }
}
public class Hierarchical {
    public static void main(String[] args) {
        C obj=new C();
        B s=new B();
    }
}
```

```
        s.sample1(10,20);
        s.sample2(10.2f);
        obj.display();
    }
}
```

■ Compilation & Execution Commands

Save the file exactly as: Hierarchical.java
javac Hierarchical.java
java Hierarchical.java

■■ Output Screen

```
This is from Super class
The sum of m and n is : 30
This is from sub class 1
The sum is : 40.2
This is from sub class 2
The product of m and n is : 0
```

■ Result

Thus, the Java program demonstrating hierarchical inheritance was successfully implemented and executed.

EXPERIMENT 14 — MULTIPLE INHERITANCE USING INTERFACES

Experiment No	14
Date	17/09/2026
Title	Multiple Inheritance using Interfaces

■ Objective / Aim

To write a Java program to demonstrate multiple inheritance using interfaces by implementing two interfaces in a single class.

■ System Requirements

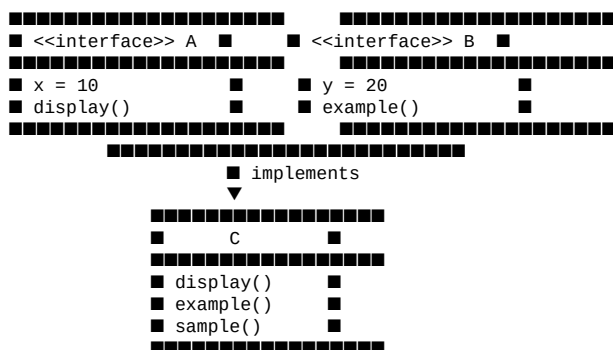
Hardware: Standard PC/Laptop, Core i3/i5 Processor, 4GB/8GB RAM.

Software: Windows/Linux/macOS, JDK 17/JDK 21, VS Code/Eclipse/IntelliJ IDEA/NetBeans.

■ Algorithm / Logic Description

1. Create interface A with display().
2. Create interface B with example().
3. Create class C implementing both A and B.
4. Implement display() and example() in C.
5. Define sample() in C.
6. Create an object of C.
7. Call all three methods and display the results.

■ UML Class Diagram



■ Source Code

```
interface A{
    int x=10;
    void display();
}
interface B
{
    int y=20;
    void example();
}
class C implements A, B
{
    public void display()
    {
        System.out.println("This is from interface A");
        System.out.println("The square of n is : " + (x*x));
    }
    public void example()
    {
        System.out.println("This is from interface B");
        System.out.println("The sum of m and n is : " + (x+y));
    }
    void sample()
    {
        System.out.println("This is from class c");
    }
}
public class Multiple {
    public static void main(String[] args) {
        C obj=new C();
        obj.display();
    }
}
```

```
        obj.example();  
        obj.sample();  
    }  
}
```

■ Compilation & Execution Commands

Save the file exactly as: Multiple.java
javac Multiple.java
java Multiple.java

■■ Output Screen

This is from interface A
The square of n is : 100
This is from interface B
The sum of m and n is : 30
This is from class c

■ Result

Thus, the Java program demonstrating multiple inheritance using interfaces was successfully implemented and executed.

EXPERIMENT 15 — HYBRID INHERITANCE USING INTERFACES

Experiment No	15
Date	17/09/2026
Title	Hybrid Inheritance using Interfaces

■ Objective / Aim

To write a Java program to demonstrate hybrid inheritance using interfaces, where interfaces form an inheritance structure and a class implements multiple interfaces.

■ System Requirements

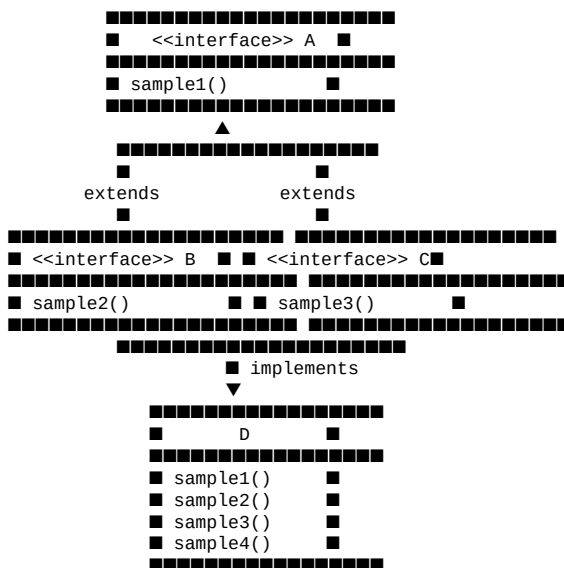
Hardware: Standard PC/Laptop, Core i3/i5 Processor, 4GB/8GB RAM.

Software: Windows/Linux/macOS, JDK 17/JDK 21, VS Code/Eclipse/IntelliJ IDEA/NetBeans.

■ Algorithm / Logic Description

1. Create interface A with sample1().
2. Create interface B extending A and add sample2().
3. Create interface C extending A and add sample3().
4. Create class D implementing B and C.
5. Implement sample1(), sample2(), and sample3().
6. Define sample4() in D.
7. Create an object of D.
8. Call all four methods and display the output.

■ UML Class Diagram



■ Source Code

```
interface A
{
    void sample1();
}
interface B extends A
{
    void sample2();
}
interface C extends A
{
    void sample3();
}
class D implements B, C {
    public void sample1()
    {
        System.out.println("THIS FROM INTERFACE A");
    }
}
```

```

    public void sample2()
    {
        System.out.println("INHERIT FROM INTERFACE B");
    }
    public void sample3()
    {
        System.out.println("INHERIT FROM INTERFACE C");
    }
    void sample4()
    {
        System.out.println("this from class D");
    }
}
public class Hybrid {
    public static void main(String[] args) {
        D obj=new D();
        obj.sample1();
        obj.sample2();
        obj.sample3();
        obj.sample4();
    }
}

```

■ Compilation & Execution Commands

Save the file exactly as: Hybrid.java
javac Hybrid.java
java Hybrid.java

■■ Output Screen

```

THIS FROM INTERFACE A
INHERIT FROM INTERFACE B
INHERIT FROM INTERFACE C
this from class D

```

■ Result

Thus, the Java program demonstrating hybrid inheritance using interfaces was successfully implemented and executed.